

Something is interesting here that we need to highlight. An E2E test, by definition, tests the whole software stack from the frontend through to the backend, including all the intermediate layers, such as databases and network infrastructure. E2E testing aims to simulate real-world scenarios and confirm that the entire application is functioning correctly.

However, when we use the `cy.intercept` function to stub HTTP requests, we are indeed modifying this behavior. We are no longer testing the complete E2E flow because we are controlling and replacing the actual backend response with a mock response. This technique transforms the test from an E2E test into something more akin to an integration test for the frontend as we are testing the integration of different components of the frontend while mocking the backend responses.

This is not necessarily a bad thing, however. Often, in testing, especially in complex systems, it is beneficial to isolate different parts of the system to gain more control over what we are testing and to ensure we can test different scenarios more reliably and deterministically.

With that, we've covered the basics of Cypress for E2E testing and you're now equipped to write robust E2E tests for your web applications.

Summary

In this chapter, we embarked on an exploration of the world of testing in a React application. We understood that the necessity of testing goes beyond mere validation of code correctness; it paves the path toward maintainability, improves readability, and drives the evolution of our application, ultimately ensuring we build software that meets expectations consistently.

Testing is a vital practice in software development – one that ensures our application not only works correctly but is also resilient to future changes. The React ecosystem, with tools such as Jest, the React Testing Library, and Cypress, provides us with a powerful arsenal to implement comprehensive testing strategies, thus bolstering the robustness and reliability of our applications.

In the next chapter, we'll look into the common refactoring techniques and see how tests can help us during the refactoring process.

6

Exploring Common Refactoring Techniques

Welcome to the fascinating world of refactoring! In this chapter, we're going to explore the basics of this fundamental practice, essential to every developer in maintaining and improving a code base. We aim to introduce you to the most common refactoring techniques, providing a solid foundation for understanding and employing these valuable tools. Remember, our objective is not to provide an exhaustive guide, but rather to familiarize you with the essentials that you'll use time and time again in your programming journey.

Refactoring doesn't discriminate among languages or frameworks – it's a universal concept applicable anywhere you write code. The techniques we'll discuss include renaming variables, changing function declarations, extracting functions, moving fields, and more. These techniques might appear simple at first glance, but they are incredibly powerful tools for crafting clean, understandable, and maintainable code.

Also remember that refactoring is not a one-time task, but rather an ongoing process of small, iterative changes that gradually enhance the structure and quality of your code. It is these frequent, incremental improvements that keep a code base healthy, robust, and easier to work with. By introducing you to the basics, we hope to equip you with the essential tools and techniques that will be a stepping stone toward more advanced refactoring methods.

While we'll delve into more complex refactoring techniques in later chapters, the practices you'll learn here will serve as a valuable starting point. By the end of this chapter, you'll have a toolkit of common refactoring practices and a newfound understanding of their importance in enhancing code quality. Ultimately, the refactoring skills you begin to develop here will empower you to write cleaner, more efficient code, and set you on the path to becoming a more proficient developer. Let's dive in!

In this chapter, we will cover the following topics:

- Understanding refactoring
- Adding tests before refactoring
- Using Rename Variable
- Using Extract Variable
- Using Replace Loop with Pipeline
- Using Extract Function
- Using Introduce Parameter Object
- Using Decompose Conditional
- Using Move Function

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch6>.

Before we dive into the refactorings, let's align with some tooling that would help us to make changes easily. When it comes to refactoring tools, there are many **integrated development environments (IDEs)** and source code editors available in the frontend world – WebStorm and Visual Studio Code (VS Code) are the most popular ones that offer an impressive range of features, including robust refactoring capabilities.

WebStorm, developed by JetBrains, is a powerful and feature-rich IDE specifically designed for JavaScript and its related technologies such as TypeScript, HTML, and CSS. One of its most notable features is its advanced automated refactoring, but it also offers an extensive list of refactoring options such as rename, extract, inline, move, and copy, which can be applied to variables, classes, functions, and other elements. It also has a smart duplication detection feature, helping you to locate and resolve repetitive blocks of code.

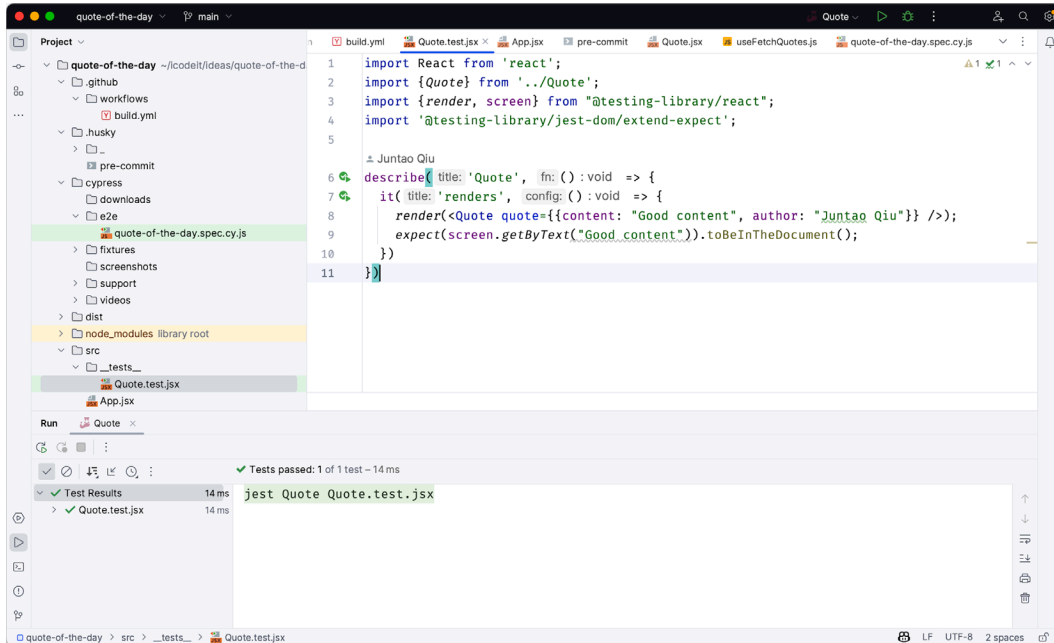


Figure 6.1: The WebStorm IDE

WebStorm's IntelliSense, auto-complete, and code navigation are quite robust, giving you a lot of help when writing and exploring the code. However, WebStorm is a commercial product, and while it does offer a trial period, you will need to purchase a license for continued use.

Visual Studio Code (VS Code), on the other hand, is a free, open source IDE developed by Microsoft. It's lightweight compared to WebStorm and is known for its speed and flexibility. VS Code also supports a wide range of languages beyond JavaScript, thanks to its extension marketplace. Refactoring capabilities in VS Code are strong as well, with support for common operations such as renaming, extracting functions or variables, and changing function signatures. VS Code's refactoring capabilities can be further enhanced by installing extensions, and its customizability is one of its key strengths.

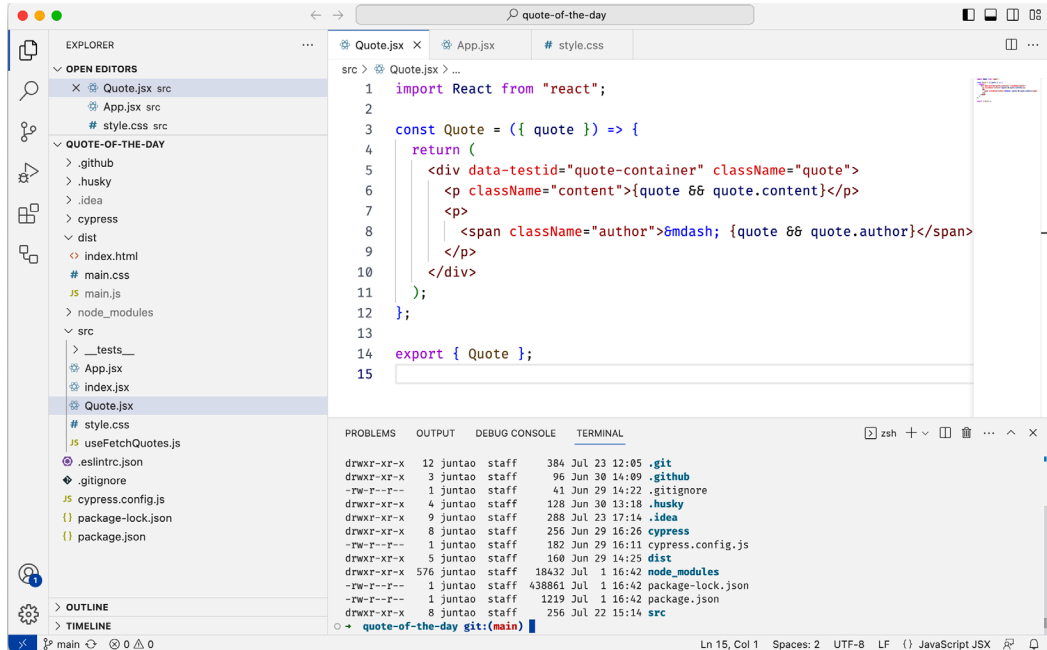


Figure 6.2: VS Code

While VS Code may not have as many automated refactoring features as WebStorm out of the box, it can be tailored to match and sometimes exceed WebStorm's capabilities through these extensions.

Choosing between the two often boils down to personal preference and the specific needs of your project. If you value a highly automated, feature-rich environment and don't mind paying for it, WebStorm might be your best bet. However, if you prioritize speed, flexibility, and customization and are comfortable setting up your environment through extensions, VS Code could be the better choice.

I prefer WebStorm as my IDE at work – part of the reason is I am already very familiar with the keymaps, and I love the built-in auto-refactoring capability. However, I still use VS Code for casual projects.

Understanding refactoring

Refactoring is a disciplined, systematic process of improving the design of an existing code base without changing its external behavior. It's a fundamental aspect of everyday coding, a practice integral to the iterative and incremental nature of software development. The concept is universally applicable, and not bound to any specific programming language, framework, or paradigm. Whether you're writing in JavaScript, Python, or any other language, and whether you're using React, Angular, or a homegrown framework, refactoring is crucial to maintaining a healthy code base.

The term “refactoring” was first introduced by William Opdyke and Ralph Johnson in a 1990 paper titled *Refactoring: An Aid in Designing Application Frameworks and Evolving Object-Oriented Systems*; however, the concept and practice of refactoring have roots in earlier practices in software engineering. The art of refactoring gained significant prominence with Martin Fowler’s book *Refactoring: Improving the Design of Existing Code*, published in 1999. In this book, Fowler describes refactoring as “a controlled technique for improving the design of an existing code base,” emphasizing its role in mitigating the buildup of technical debt, which makes the code easier to understand, maintain, and extend.

Refactoring isn’t about making one grand, sweeping change to perfect the code base. Instead, it’s about making small, incremental improvements consistently over time. Each individual change might not dramatically alter the quality of the code, but collectively, over time, these small changes can significantly enhance the structure, readability, and maintainability of the code base.

Although refactoring doesn’t add new functionality, it directly influences the team’s ability to deliver new features more quickly, with fewer bugs, and to respond more flexibly to changing requirements. By continuously refactoring, we keep our code clean and easy to work with and set the stage for long-term, sustainable development.

In conclusion, refactoring is a critical tool in a developer’s toolkit, irrespective of the technology stack or the size and scope of the project. It’s a long-term investment in the code base and the team, and ultimately, it’s an investment in the quality of the software that is delivered.

The common mistakes of refactoring

The biggest mistake people make in refactoring is that they restructure code rather than refactoring it. The terms “refactoring” and “restructuring” are often used interchangeably, but they have distinct meanings in software development.

Refactoring is a disciplined technique for improving the design of an existing code base, making it cleaner and easier to understand and work with. It involves changing the internal structure of the software without modifying its external behavior. This is generally done in small steps, and each refactoring step is expected to maintain the software’s functionality. It doesn’t add new features; instead, it makes the code more readable, maintainable, and prepared for future changes.

For example, in a React application, refactoring could involve breaking down a large component into smaller, more manageable components, or replacing complex conditional logic with a strategy pattern.

Restructuring, on the other hand, can be seen as a broader and more drastic process. It often involves large-scale changes that not only impact the internal structure of the software but can also affect its external behavior. Restructuring can encompass changes to the software’s architecture, data models, interfaces, and more. It is often driven by the need to introduce major changes or additions to the software’s features or capabilities, to improve performance, or to address significant technical debt.

In the context of a React application, restructuring might involve changing the state management solution (such as moving from Redux to the React Context API), updating the routing mechanism, or transitioning from a monolithic architecture to a microfrontend architecture.

While both refactoring and restructuring aim to improve the quality of the code base, refactoring is typically smaller in scope, involves no change in functionality, and is part of the regular development process. In contrast, restructuring is generally larger in scope, can change functionality, and is often part of a larger project or initiative to address more significant challenges or changes in requirements.

As well as a misunderstanding between refactoring and restructuring, another mistake people tend to make is that they don't test as often – sometimes this is because they don't have many tests, while at other times they think it's safe to make these “small” changes without testing. Let's look at testing in the next section.

Adding tests before refactoring

Because we don't want to make any observable behavior changes during refactoring, we need to inspect the code to make sure we have enough tests to cover the current behavior. It's easy to mess up without the right tests in place, and that's not only risky but also less efficient, as we need to check the changed code manually and repeatedly.

Let's say we have some TypeScript code from an online shopping application – the code works fine, but there aren't any tests associated with it. To improve the code so that it's easier to understand and extend, we need to refactor it:

```
interface Item {
  id: string;
  price: number;
  quantity: number;
}

class ShoppingCart {
  cartItems: Item[] = [];

  addItemToCart(id: string, price: number, quantity: number) {
    this.cartItems.push({ id, price, quantity });
  }

  calculateTotal() {
    let total = 0;
    for (let i = 0; i < this.cartItems.length; i++) {
      let item = this.cartItems[i];
      let subTotal = item.price * item.quantity;
      if (item.quantity > 10) {
```

```
        subTotal *= 0.9;
      }
      total += subTotal;
    }
    return total;
  }
}

export { ShoppingCart };
```

So, this code defines a shopping cart model. Firstly, it defines an `Item` interface, which represents an item to be added to the cart. An `Item` component consists of an ID, a price, and a quantity. Then, it defines a `ShoppingCart` class with a `cartItems` property, which is an array of `Item` objects, initially empty.

The `ShoppingCart` class has two methods:

- The `addItemToCart` method accepts an ID, price, and quantity, and then creates an item using these parameters. This item is then added to the `cartItems` array.
- The `calculateTotal` method calculates the total price of the items in the cart. For each item, it multiplies the item's price by its quantity to get a subtotal. If the quantity of the item is more than 10, a 10% discount is applied to the subtotal. The subtotal of each item is then added together to get the total. The total is then returned by the method.

There are two important calculations here: compute the total price (price by quantity) and apply the discount when eligible. We normally should pay more attention to the logic regarding these calculations.

For example, we need to verify both sides of an `if-else` statement. As we have `if-else` inside the `for` loop, we will at least need to add two test cases before making changes. Let's add the following Jest tests to describe both calculations – with and without a discount:

```
import { ShoppingCart } from "../ShoppingCart";

describe("ShoppingCart", () => {
  it("calculates item prices", () => {
    const shoppingCart = new ShoppingCart();
    shoppingCart.addItemToCart("apple", 2.0, 2);
    shoppingCart.addItemToCart("orange", 3.5, 1);

    const price = shoppingCart.calculateTotal();
    expect(price).toEqual(7.5);
  });

  it('applies discount when applicable', () => {
```



```
const shoppingCart = new ShoppingCart();
shoppingCart.addItemToCart("apple", 2.0, 11);

const price = shoppingCart.calculateTotal();
expect(price).toEqual(19.8);
  })
});
```

The first test, `calculates item prices`, is verifying that the `calculateTotal` method works as expected when no discounts are applied. Here, a `ShoppingCart` object is instantiated, and two items (apple and orange) are added to the cart. The total price of these items is calculated and expected to be 7.5, as there are two apples at \$2 each and one orange at \$3.5.

The second test, `applies discount when applicable`, is checking that the `calculateTotal` method correctly applies a 10% discount when the quantity of an item is more than 10. In this case, a `ShoppingCart` object is instantiated, and one type of item (apple) is added to the cart with quantity 11. The total price of this item should be \$19.8 after applying the 10% discount on the subtotal of \$22 (11 apples at \$2 each). The calculated total price is then checked against this expected value.

Once we have test cases to cover the important logic, we can safely make changes. During refactoring, we'll need to run these tests regularly.

Refactorings are small steps that can improve the code. Let's have a look at our very first and maybe the simplest refactoring technique: **Rename Variable**.

Using Rename Variable

Let's start with a simple refactoring technique called Rename Variable. Rename Variable is a very straightforward yet effective method to improve the readability and maintainability of the code. It involves changing the name of a variable to better reflect its purpose and the data it holds, or to follow a certain naming convention or standard.

Sometimes, during the initial phases of coding, developers might choose names for variables that make sense at that time, but as the code evolves, the purpose of the variable might change or become clearer. The variable name, however, often remains the same. This can cause confusion and make the code harder to understand and maintain. Renaming variables to describe their purposes more accurately can reduce the cognitive load for future readers of the code, including the future self of the current coder.

Let's return to our `ShoppingCart` example. The variable name `cartItems` inside the `ShoppingCart` class is a little redundant; however, we can rename it to simply `items` to be more concise and clean:

```
class ShoppingCart {
  items: Item[] = [];

  addItemToCart(id: string, price: number, quantity: number) {
```

```
    this.items.push({ id, price, quantity });
  }

  calculateTotal() {
    let total = 0;
    for (let i = 0; i < this.items.length; i++) {
      let item = this.items[i];
      let subTotal = item.price * item.quantity;
      if (item.quantity > 10) {
        subTotal *= 0.9;
      }
      total += subTotal;
    }
    return total;
  }
}
```

After the change, make sure to run the tests again to see whether we accidentally made any mistakes.

It's important to establish a habit of running tests regularly after making some changes, and whenever the tests fail, we need to stop and examine what is wrong. Once all the tests are back to passing, we can then proceed.

Using Extract Variable

Extract Variable is a common refactoring technique that is used to improve the readability and maintainability of code. The process involves taking a section of code that calculates a value, replacing it with a new variable, and assigning the result of the original expression to this new variable. There is a similar refactoring called **Extract Constant** that can be used to extract a value that doesn't change at runtime.

This refactoring technique is particularly useful when you have a complex expression or a duplicated calculation; by extracting parts of the expression to variables with meaningful names, the code becomes more understandable and easier to manage.

In the `ShoppingCart` example, the `0.9` discount rate deserves its own name; we can extract a variable and reference it at the point where the function is called. As the value of the variable isn't going to change at runtime, we can call it **Extract Constant** in this case:

```
const DISCOUNT_RATE = 0.9;

class ShoppingCart {
  //...
  calculateTotal() {
    let total = 0;
```

```
    for (let i = 0; i < this.items.length; i++) {  
      let item = this.items[i];  
      let subTotal = item.price * item.quantity;  
      if (item.quantity > 10) {  
        subTotal *= DISCOUNT_RATE;  
      }  
      total += subTotal;  
    }  
    return total;  
  }  
  //...  
}
```

For the sake of clarity, portions of the code that aren't relevant to this particular change have been left out of the discussion; however, the important thing to note is that in this instance, we've created a constant named `DISCOUNT_RATE` and used it in place of the previous hard-coded value of `0.9` where it's utilized in the code. At times, we may wish to assign a name to an expression other than a hard-coded value, so we can create a variable to stand in for the expression and then refer to that variable.

It's a tiny step, but it improves the code slightly. If we need to change the discount rate in the future, the constant name is much easier to search and understand than the hard-coded value of `0.9`.

Now we can investigate another refactoring technique to make the `for` loop a bit simpler.

Using Replace Loop with Pipeline

The **Replace Loop with Pipeline** refactoring technique, as the name implies, is about replacing a loop structure with a pipeline of transformations, commonly achieved by using higher-order functions or methods such as `map`, `filter`, and `reduce` in functional programming languages such as JavaScript.

In the case of JavaScript, the Array prototype has methods such as `map`, `filter`, and `reduce` that can be chained together to form a pipeline. Each of these methods receives a function as an argument and applies this function to each element in the array, effectively transforming the array in some manner.

However, keep in mind that while replacing loops with pipelines can make the code cleaner and more readable, it might not always be the most efficient option, especially when dealing with very large datasets. So, as with all refactorings, you need to balance readability and maintainability with performance requirements in cases when you need to iterate through a large dataset multiple times.

The `for` loop in the previous section can be replaced by the `reduce` function (and we don't have to explicitly define an index variable or save the boilerplate code):

```
class ShoppingCart {  
  //...  
  calculateTotal() {
```

```
    return this.items.reduce((total, item) => {
      let subTotal = item.price * item.quantity;
      return total + (item.quantity > 10 ? subTotal * DISCOUNT_RATE :
        subTotal);
    }, 0);
  }
  //...
}
```

The `calculateTotal()` method is using the `reduce()` function to calculate the total price of the items in the shopping cart. The `reduce()` function is a higher-order function that applies a function against an accumulator and each element in the array (from left to right) to reduce it to a single output value.

The total starts at 0 and then for each item in the shopping cart, it adds the `subTotal` variable of that item to the total. The `subTotal` variable is calculated by multiplying the price and quantity of each item.

Next, we need to re-run all the tests to check whether everything goes well. As our tests are still passing, let's see how we can make the code even better by extracting lines into a smaller function.

Using Extract Function

Extract Function is a refactoring technique that helps to improve code readability and maintainability by breaking down a large or complex function into smaller, more manageable parts.

Let's say you come across a function that's performing multiple tasks. Maybe it's doing some data validation, then some computations, and finally logging the result or updating some state. The function is long and complex, making it hard to understand at a glance what it's doing. Extract Function refactoring is all about identifying those distinct pieces of functionality, pulling them out into their own separate functions, and then calling those new functions from the original one.

One key benefit is that it makes the code more self-documenting. If you extract a part of your function to a new function and give it a meaningful name, it can often make the code much easier to understand since the function name can describe what the code is doing. It also improves the reusability of the code, since these smaller functions can be reused elsewhere if needed.

The logic of how to calculate the `subTotal` variable can be extracted from `calculateTotal` as a separate unit:

```
function applyDiscountIfEligible(item: Item, subTotal: number) {
  return item.quantity > 10 ? subTotal * DISCOUNT_RATE : subTotal;
}

class ShoppingCart {
```

```
//...

calculateTotal() {
  return this.items.reduce((total, item) => {
    let subTotal = item.price * item.quantity;
    return total + applyDiscountIfEligible(item, subTotal);
  }, 0);
}
```

In this code snippet, we see the result of an Extract Function refactoring. The logic to apply a discount if the item quantity is greater than 10 has been extracted into its own function named `applyDiscountIfEligible`.

In the `ShoppingCart` class, the `calculateTotal` method calculates the total price of the items in the cart using the `reduce` function. For each item, it computes the subtotal as the product of the item's price and quantity, then adds this subtotal (after applying any eligible discount) to the total.

The `applyDiscountIfEligible` function takes an item and its quantity as arguments. If the quantity of the item is more than 10, it applies a discount rate (represented by `DISCOUNT_RATE`) to the argument `subTotal`; otherwise, it simply returns the argument `subTotal` as it is.

This refactoring makes the `calculateTotal` method more concise and easier to read, by abstracting away the details of how discounts are applied into a separate, appropriately named function.

Let's look at another refactoring method that could make the passed-in parameter easy to modify.

Using Introduce Parameter Object

Introduce Parameter Object is a refactoring technique used when a function has a large number of parameters, or when multiple functions share the same parameters. In this technique, you group related parameters into a single object and pass that object to the function instead.

A large number of parameters in a function can be confusing and difficult to manage. Grouping related parameters together into an object can increase code readability and make it easier to understand what the function does. It also makes the function call simpler and cleaner. Furthermore, if the same group of parameters is used in multiple function calls, this technique reduces the chances of passing parameters in the wrong order.

For example, consider a `calculateTotalPrice(quantity, price, discount)` function. We could refactor this using the Introduce Parameter Object technique to become

`calculateTotalPrice({ quantity, price, discount })`. Now, the `quantity`, `price`, and `discount` parameters are grouped together into an object (with type `Item`), like so:

```
class ShoppingCart {
  items: Item[] = [];

  addItemToCart({id, price, quantity}: Item) {
    this.items.push({ id, price, quantity });
  }

  //...
}
```

On top of these benefits, Introduce Parameter Object refactoring can often reveal or inspire domain concepts that were previously hidden and implicit in your code. The parameter object might become a class of its own, with its own behavior and data manipulation methods. This can lead to more object-oriented and encapsulated code.

Next, let's explore another refactoring technique designed to streamline your `if-else` statements and enhance code readability.

Using Decompose Conditional

Decompose Conditional is a refactoring technique where the logic within a conditional statement (`if-else` or `switch`) is extracted into separate functions. This technique helps to improve the readability of the code, making it more understandable.

The condition, `if` clause, and `else` clause (if it exists) all get their own function. These functions are then named according to what they do or what they are checking for. This refactoring is beneficial because it replaces code that might need comments to understand with well-named functions, making the code self-explanatory.

For example, the logic in the `applyDiscountIfEligible` function can actually be simplified by this refactoring; we can extract a small function called `isDiscountEligible` to replace the `item.quantity > 10` check, like in the following:

```
function isDiscountEligible(item: Item) {
  return item.quantity > 10;
}

function applyDiscountIfEligible(item: Item, subTotal: number) {
  return isDiscountEligible(item) ? subTotal * DISCOUNT_RATE :
  subTotal;
}
```

In this code snippet, the extraction of logic into a separate function might appear superfluous because it adds an additional function call. However, it enhances readability and reusability:

```
function isDiscountEligible(item: Item) {  
    return item.quantity > 10;  
}  
  
function applyDiscountIfEligible(item: Item, subTotal: number) {  
    return isDiscountEligible(item) ? subTotal * DISCOUNT_RATE :  
        subTotal;  
}
```

In this code snippet, we've separated the logic that determines whether an item is eligible for a discount into a standalone `isDiscountEligible` function. This extraction makes our `applyDiscountIfEligible` function cleaner and its intention more evident. Additionally, it allows for the `isDiscountEligible` logic to be updated independently if needed in the future, improving maintainability.

After extracting these smaller functions, they don't have to reside in the current file. We can relocate them to a separate module and import them as needed; this not only shortens the length of the current module but also improves its readability. Let's look at that next.

Using Move Function

Move Function is a refactoring method that involves changing the location of a function to a more suitable or appropriate place. This could be within the same class, to a different class, or even to a separate module. The objective of this method is to enhance the readability, maintainability, and structure of the code by ensuring that functions are placed where they logically fit best.

This kind of refactoring becomes necessary when the responsibilities of your classes evolve over time. You might find that a function makes more sense in a different class, or perhaps you have a group of functions within a class that work together and would be better suited in their own class or module.

Move Function refactoring can help reduce the complexity of the class by moving functions to the places where their functionality is most relevant or required. This promotes the principle of cohesion where related code is kept together. It also aids in achieving loose coupling by minimizing unnecessary dependencies between different parts of the code.

In our `ShoppingCart` component, we can move the type definition into a new file called `types.ts`. We can also move `DISCOUNT_RATE`, `isDiscountEligible`, and `applyDiscountIfEligible` into a separate file called `utils.ts`:

```
import { Item } from "../types";

const DISCOUNT_RATE = 0.9;

function isDiscountEligible(item: Item) {
  return item.quantity > 10;
}

export function applyDiscountIfEligible(item: Item, subTotal: number) {
  return isDiscountEligible(item) ? subTotal * DISCOUNT_RATE :
    subTotal;
}
```

Note that in the code, only `applyDiscountIfEligible` is a public function and can be accessed outside of the file. This refactoring also improves the encapsulation of the code.

After Move Function is used, the `ShoppingCart` component is simplified significantly and only has the necessary parts:

```
import { Item } from "../types";
import { applyDiscountIfEligible } from "../utils";

class ShoppingCart {
  items: Item[] = [];

  addItemToCart({ id, price, quantity }: Item) {
    this.items.push({ id, price, quantity });
  }

  calculateTotal() {
    return this.items.reduce((total, item) => {
      let subTotal = item.price * item.quantity;
      return total + applyDiscountIfEligible(item, subTotal);
    }, 0);
  }
}

export { ShoppingCart };
```


As with all refactoring, care should be taken when moving functions to ensure that the overall behavior of the system is not altered. Tests should be in place to verify that the functionality remains the same after refactoring.

Summary

This chapter focused on a variety of code refactoring techniques that are essential to maintaining and improving the structure, readability, and maintainability of your code base.

The refactoring techniques introduced include Rename Variable, which enhances code clarity by using more descriptive variable names, Extract Variable, which simplifies complex expressions by breaking them into smaller, more manageable parts, and Replace Loop with Pipeline, which transforms traditional `for/while` loops into more concise, declarative higher-order functions such as `map`, `filter`, and `reduce`.

As well as this, Extract Function encourages code modularity and reusability by breaking down large functions into smaller ones, each with a single, well-defined responsibility, while Introduce Parameter Object groups related parameters into a single object, thereby reducing the complexity of function signatures. Plus, Decompose Conditional breaks down complex conditional logic into separate functions, enhancing readability, and Move Function ensures that functions are placed in the most logical and appropriate location in your code base, promoting high cohesion and loose coupling.

Throughout all these techniques, we emphasized the importance of maintaining the same overall system behavior and relying on tests to ensure that functionality remains consistent despite refactoring. These methods, when properly applied, can lead to a more understandable, easier to maintain, and more robust code base.

In the next chapter, we will explore an exceptional approach to enhancing the quality of our code – the method known as test-driven development.

Introducing Test-Driven Development with React

Welcome to a chapter that could potentially revolutionize your approach to React development – **test-driven development** (or **TDD** for short). If you’ve been building React applications, you know how complex and intricate they can get. With various states to manage, components to juggle, and user interactions to facilitate, ensuring the reliability of your code base can be challenging. That’s where TDD comes in.

In the ever-changing landscape of software development, where features are continuously added or modified, TDD serves as a lighthouse that guides you safely through the rough seas of bugs and regressions. By writing tests before your actual code, you not only confirm that your code does what it’s supposed to, but you also create a safety net that makes future changes less risky.

This chapter aims to deepen your understanding of TDD and how to implement it effectively in React applications. We’ll introduce the core principles of TDD, explore various styles, including unit test-driven development, **acceptance test-driven development (ATDD)**, and **behavior-driven development (BDD)**, and even examine the nuanced differences between the Chicago and London styles of TDD.

But we won’t stop at theory – to make these concepts come alive, we’ll walk you through a practical example of creating a pizza store menu page. From setting up the initial structure to managing complex features, we’ll guide you through each step with the TDD approach. By the end of this chapter, you’ll have a firm grasp of TDD’s capabilities, and you’ll be well equipped to start writing more reliable, robust React applications.

So, get ready to dive into a world where tests lead the way and code follows, creating a harmonious balance that results in better, more reliable software.

In this chapter, we will cover the following topics:

- Understanding TDD
- Introducing tasking
- Introducing the online pizza store application
- Breaking down the application requirements
- Implementing the application headline
- Implementing the menu list
- Creating the shopping cart
- Adding items to the shopping cart
- Refactoring the shopping cart

Technical requirements

A GitHub repository has been created to host all the code we'll discuss in this book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch7>.

Understanding TDD

TDD isn't exactly a new kid on the block. Originating from **Extreme Programming (XP)**, a software development methodology that encourages frequent releases in short cycles, TDD has roots going back to the late 1990s. It was Kent Beck, one of the original signatories of the Agile Manifesto, who popularized this practice as a core part of XP. The practice has since grown beyond the realm of XP and is now commonly utilized in various methodologies and frameworks, React included.

At the heart of TDD is a very simple, yet profoundly effective cycle known as the **Red-Green-Refactor loop**:

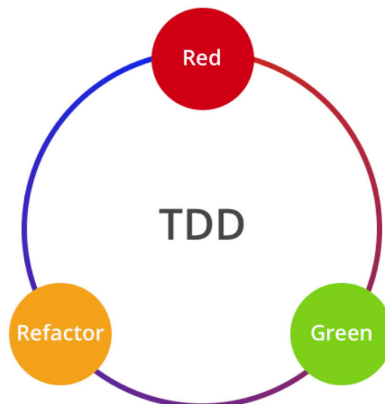


Figure 7.1: The Red-Green-Refactor loop

As you can see, there are essentially three steps when practicing TDD:

- **Red:** At this stage, you write a test that defines a function or improvements of a function. This test should initially fail because the function isn't implemented yet. In most test frameworks (for example, Jest), there will be some red text to indicate the failure.
- **Green:** At this stage, you write the minimum amount of code necessary to pass the test. The key here is to write as little code as possible to make the test pass and make the text turn green – no more than that.
- **Refactor:** Finally, you need to clean up the code while keeping it functional. The refactor phase is about making the code efficient, readable, and understandable without changing its behavior. The tests that are written should still pass after the refactoring.

When developers first encounter TDD, it often feels counter-intuitive, as the practice of writing tests before the actual code contradicts traditional development instincts. However, once you get past the initial discomfort, the advantages of TDD become hard to ignore:

- **Focused problem-solving:** By writing a test for a specific functionality first, you focus your attention solely on solving one problem at a time, making the development process less overwhelming.
- **Predictable next steps:** When you follow a test-driven approach, you always know what to do next: make the test pass. This reduces the cognitive load, making it easier to focus on the task at hand.
- **Simple, maintainable design:** The process naturally encourages the simplest code necessary to pass tests, resulting in a design that is as minimal as possible, and thereby easier to understand and maintain.
- **Facilitates mental flow:** The loop provides a structured approach to coding that helps maintain a “mental flow,” helping you stay focused on tasks by reducing the constant context-switching that interrupts a productive coding session.
- **Automatic test coverage:** TDD ensures that your application has robust test coverage by default. You're not adding tests as an afterthought; they're integral to the development process, ensuring a more stable codebase.

TDD is a practice deeply rooted in the principles of Agile and XP but has found relevance far beyond those methodologies. With its structured Red-Green-Refactor loop, TDD provides a solid framework for writing high-quality code. Although it may appear counter-intuitive initially, adopting TDD can result in more focused problem-solving, predictable development, simpler design, enhanced productivity, and robust test coverage.

Different styles of TDD

The core principles of TDD have been adapted and extended into various styles, each offering different perspectives on how best to approach testing and development. Let's explore some of these styles to understand how they can be applied to React development.

The original form of TDD, simply referred to as TDD, mainly focuses on unit tests. In this style, you write tests for the smallest pieces of your code – often individual methods or functions. The aim is to ensure that each part of your code base works as expected in isolation. While this is powerful for testing logic and algorithms, it may not fully capture how various parts interact, especially in a complex UI framework such as React.

ATDD extends TDD by beginning the development process with user acceptance tests. This means that before writing any code, you define what “done” looks like from a user's perspective, often in collaboration with stakeholders. These acceptance tests are then used as the foundation for developing features. ATDD is particularly useful for ensuring that you're building what the user needs and wants:

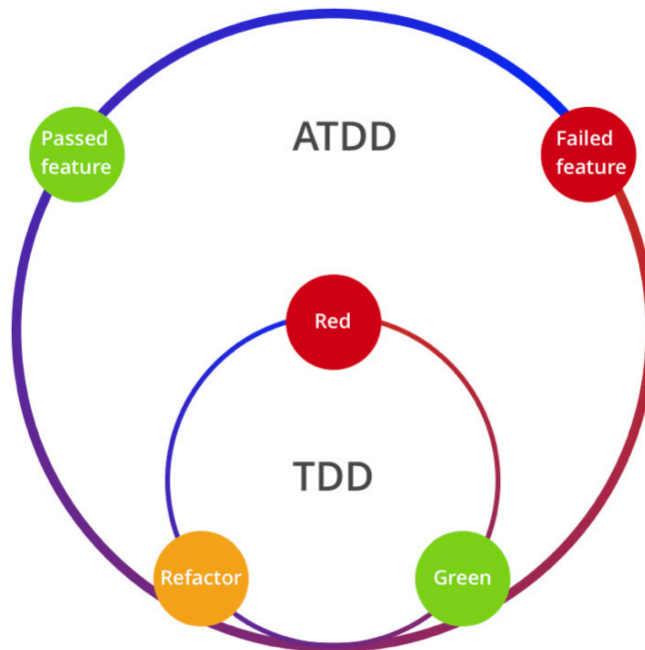


Figure 7.2: The ATDD loop

Note that when you write an acceptance test, it can usually be broken down into smaller unit tests. For example, a user logging into the system can be an acceptance test, but there will be forgotten passwords, incorrect passwords or usernames, remember me functions, and more to cover in the lower-level unit tests.

BDD is a further refinement of TDD and ATDD, focusing on the behavior of an application for a given input. Rather than writing tests that check if a particular method returns an expected value, BDD tests check if a system behaves as expected when subjected to certain conditions. BDD often uses more descriptive language to define tests, making it easier for non-technical stakeholders to understand what is being tested.

BDD often makes use of tools such as **Cucumber** to define behavior specifications in a human-readable format. In a Cucumber test, you specify behavior using a plaintext language called **Gherkin**. Here's a simple example of a BDD test case using Cucumber for a pizza ordering feature (we will continue working on this pizza example later in this chapter):

```
Feature: Pizza Ordering

  Scenario: Customer orders a single pizza
    Given I'm on the PizzaShop website
    When I select the "Order Pizza" button
    And I choose a "Margherita" pizza
    And I add it to the cart
    Then the cart should contain 1 "Margherita" pizza

  Scenario: Customer removes a pizza from the cart
    Given I'm on the PizzaShop website
    And the cart contains 1 "Margherita" pizza
    When I remove the "Margherita" pizza from the cart
    Then the cart should be empty
```

This Gherkin file defines the expected behavior of the pizza ordering functionality. Each line is called a step and can be interpreted as a statement in the test. The scenarios describe the test's behavior in terms of the steps to be performed and the expected outcomes.

Gherkin syntax is more than just readable documentation – it's executable. Tools such as Cucumber can parse the Gherkin files and execute tests based on them – for example, **Given I'm on the PizzaShop website** is transformed into `cypress.visit("http://pizzashop.com")`. This ensures that the software behaves exactly as described in the feature files, making it a source of truth that evolves along with the application.

BDD feature files (Gherkin) serve as a form of living documentation that gets updated as the application changes. This makes them incredibly valuable for new team members, or even for seasoned developers, to understand the expected behavior of the application quickly.

Focusing on user value

Regardless of the style you choose, when working with React, it's crucial to focus on the user's perspective. React components are pieces of UI that users interact with, so your tests should reflect that interaction. The user doesn't care about how your state is managed or how efficient your life

cycle methods are; they care about whether clicking a button shows a dropdown, or whether a form submission produces the expected result.

Kent C. Dodds, the creator of the React Testing Library, said “*The more your tests resemble the way your software is used, the more confidence they can give you.*” This principle is universally applicable, irrespective of the framework or library you’re using. The focus should always be on the user’s experience.

This user-centric approach aligns well with BDD and ATDD, where the focus is on the result of an interaction, not the minutiae of the implementation. By adhering to these principles, you can ensure your React components not only work well but also deliver the user experience you aim to achieve.

Now that we’ve understood what TDD is and how its various styles can aid in delivering value to our customers, the next question to tackle is: how do we go about implementing it?

Introducing tasking

Tasking is an essential step in the TDD process that involves breaking down a feature or user story into small, manageable tasks, which then serve as the basis for your test cases. The goal of tasking is to create a clear roadmap for what you’re going to code, how you’ll test it, and in what order you’ll proceed.

Breaking the big requirement into smaller chunks has a lot of benefits:

- **It clarifies scope:** Breaking down a feature into tasks helps establish a better understanding of what needs to be done and how to approach it
- **It simplifies the problem:** By dissecting a complex problem into smaller tasks, you make it easier to tackle
- **It prioritizes work:** Once the tasks have been laid out, they can be prioritized to deliver the most value first or to build logically upon one another
- **It focuses effort:** Tasking ensures that each test you write serves a clear, immediate purpose, making your TDD cycle more efficient
- **It facilitates collaboration:** Team members can pick up individual tasks, safe in the knowledge that they are all contributing to a cohesive whole

Now, you might be wondering, if it is so good and helpful, how can we do tasking? It’s not anything fancy – you might have already done it without noticing. You just follow these steps:

1. **Review the user story or requirement:** Understand the user story or feature you are supposed to implement.
2. **Identify logical components:** Break the story down into its logical components, which often correspond to domain concepts, business rules, or individual steps in a user workflow.

3. **Create a task list:** Write down a list of tasks. These tasks should be small enough that you can write a few test cases and the corresponding implementation code in a short amount of time (say, 15 to 30 minutes).
4. **Sequence the tasks:** Determine the most logical order for completing these tasks, often starting with the “happy path” – the default scenario where everything goes as expected, without encountering any errors – and then moving on to edge cases and error handling.
5. **Map the tasks to tests:** For each task, identify the tests that will verify that part of the functionality. You don’t need to write the tests at this stage; you’re simply identifying what they’ll be.

Tasking might be a part of your daily workflow without you even realizing it. It’s a systematic approach to problem-solving that involves breaking down a requirement into manageable, sequential tasks. These tasks should ideally be completable within a few minutes to an hour.

The process of TDD is akin to the art of painting. You begin with a sketch or draft, outlining basic shapes and lines with a pencil, much like framing the initial structure of your code. In the beginning, the vision might be vague, a mere idea or concept in your mind. But as you draw – or write tests and code – the image begins to take shape. More elements are added, details emerge, and adjustments are made, allowing for continuous refinement. With each layer or iteration, clarity emerges, yet the exact final appearance remains a mystery until the very last stages. Just as an artist crafts a masterpiece through gradual development, TDD shapes a robust and elegant piece of software.

All right, we have covered a lot of theory so far. Let’s look into a concrete example to fully understand how to do tasking and use tasks as a guideline for applying the Red-Green-Refactor loop.

Introducing the online pizza store application

In this section, we’ll be diving into the TDD process through a deliciously practical example: *The Code Oven*, an online pizza store. Named to celebrate the fusion of coding and culinary arts, The Code Oven aims to serve both your appetite and your intellectual curiosity. This digital storefront will offer us a comprehensive sandbox where we can apply all the TDD principles and techniques we discuss.

Here’s what you can expect to see in The Code Oven:

- **Pizza menu:** At the heart of The Code Oven is an appetizing menu of eight delectable pizzas. Each pizza is presented with its name and cost, designed to whet your appetite and inform your choice.
- **An Add button:** Beside each mouth-watering option is an **Add** button. This enables users to start the ordering process by adding their chosen pizzas to a virtual shopping cart.
- **Shopping cart:** A designated section on the screen shows users their current shopping cart, complete with the names and prices of each selected pizza.

- **Modify cart:** Should you get second thoughts or just want more pizza, The Code Oven allows you to modify your cart by adding or removing items dynamically.
- **Order total:** There's no need for manual calculations – The Code Oven's cart automatically computes and displays the total price of your chosen items.
- **A Place my order button:** A prominent **Place my order** button serves as the final step, which would, in a real-world application, process the order for delivery or pick-up:

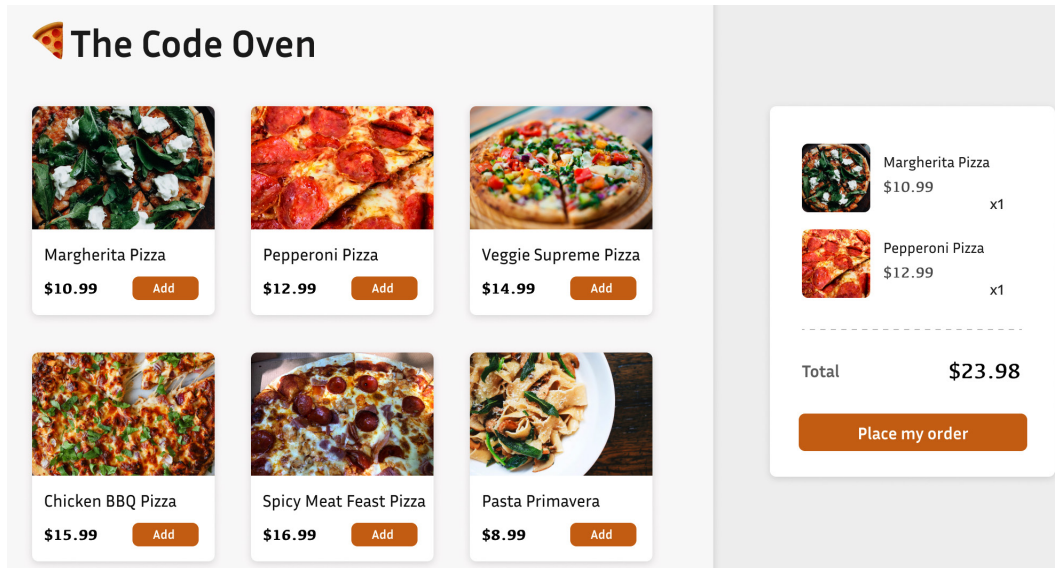


Figure 7.3: The Code Oven

As we build out The Code Oven, we'll be applying TDD at every stage to ensure that our virtual pizzeria is not just functional but robust and easily maintainable. Get ready to roll up your sleeves, both for coding and for some virtual pizza-making!

Breaking down the application requirements

There isn't a universal right way to break down the requirements of the application you are making; however, there are typically two different styles – the bottom-up style and the top-down style.

In the **bottom-up** style of TDD, developers start by writing tests and implementing functionality for the smallest and most fundamental components of the system. This approach emphasizes the construction of individual units or classes, thoroughly testing them before integrating them into higher-level components. It provides strong validation for the underlying parts of the system and helps in creating a robust foundation.

However, this style might lead to challenges in integrating the components if the bigger picture and interactions between the units are not carefully considered.

Back to the online pizza store, we can break the whole page down into the following tasks:

- Implement a single `PizzaItem` component with a pizza name
- Add a price to `PizzaItem`
- Add a button to `PizzaItem`
- Implement a `PizzaList` component (render three items in a row, for example)
- Implement a simple `ShoppingCart` component with a button
- Support the ability to add/remove items to the `ShoppingCart` component
- Add a calculation for the total number of pizzas
- Implement the whole application with these individual standalone components

As you can see, each task focuses on an individual component at a time. The components start simple, with only the minimal function; then, we incrementally add more features to them, including test cases to cover the functionality required, as well as other reasonable edge cases.

So, we start from a single `PizzaItem` component (which only has a name in it) and then give it a price, and then a button. After the individual items are built, we start to implement `PizzaList`, and then `ShoppingCart`. Then, once `PizzaList` and `ShoppingCart` are done, we integrate them and test a few overall functions from the user's perspective.

For example, as demonstrated in the following screenshot, we might start with a `PizzaItem` component, implementing the component gradually without worrying about anything else in the application. Once we have a full implementation of `PizzaItem` (with an image, name, price and **Add** button), we can move on to the next component, `ShoppingCart`:

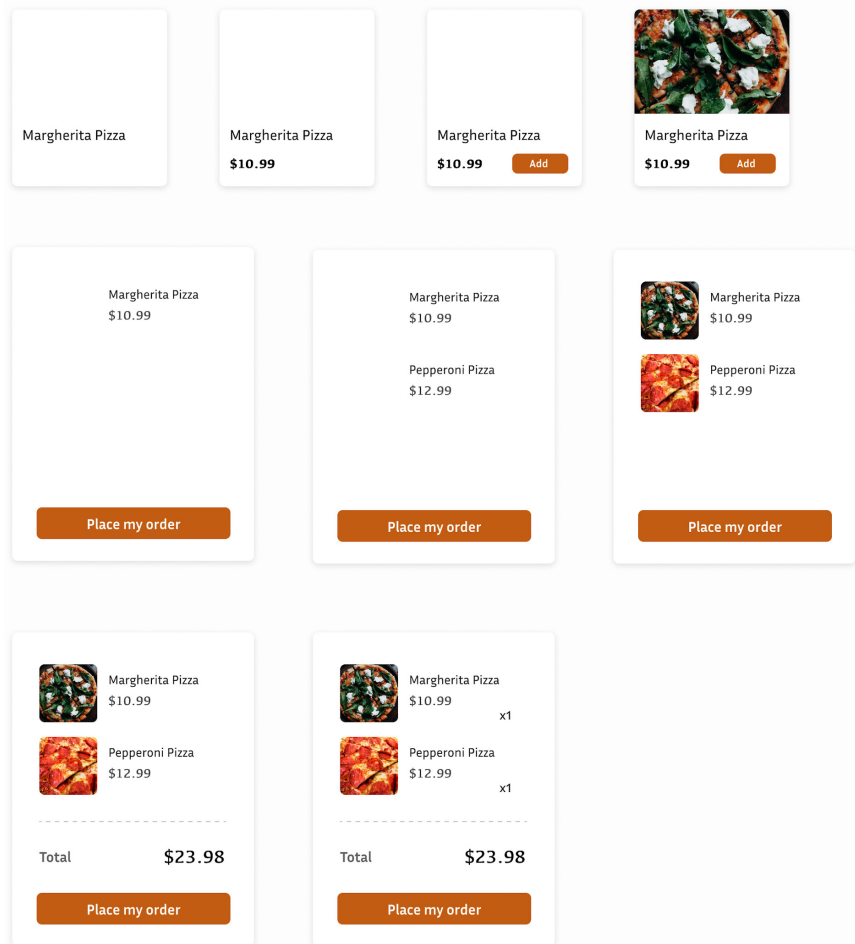


Figure 7.4: The bottom-up style

The **top-down** style of TDD takes the opposite approach, beginning with the high-level architecture and overall functionality of the system. Developers first write tests and implement features for the main components and then gradually work their way down to the more detailed and specific functionalities.

This style helps ensure that the system's primary objectives and workflow are established early on, providing a clear roadmap for the development process. It can foster better integration and alignment with the overall goals but may sometimes require the use of temporary “stubs” or “mocks” to simulate lower-level components before they are developed. For instance, we could break the feature down into the following list:

- Implement the page title
- Implement a menu list containing the pizza names
- Implement a `ShoppingCart` component with only a button (disabled by default)

- Add an item to ShoppingCart when the button is clicked, after which the ShoppingCart button is enabled
- Add a price to the ShoppingCart component
- Add a total number of selected items to ShoppingCart
- Remove an item from ShoppingCart, with the total number changing accordingly

For the top-down approach, we don't have a clear picture of the individual units but the working application as a whole – so we see the application from outside without knowing the implementation details.

For example, there isn't a `PizzaItem` component at the beginning, and the smaller components are gradually extracted from the bigger component when we find that the component is too big. This means we will always have functional software running (even if we don't have the small well-designed components up-front), allowing us to stop at any time without breaking the functionality.

A possible breakdown with the top-down approach is shown in the following diagram. We start with an empty list, then a list with pizza names, and then a shopping cart that allows users to add items and proceed to the next step:

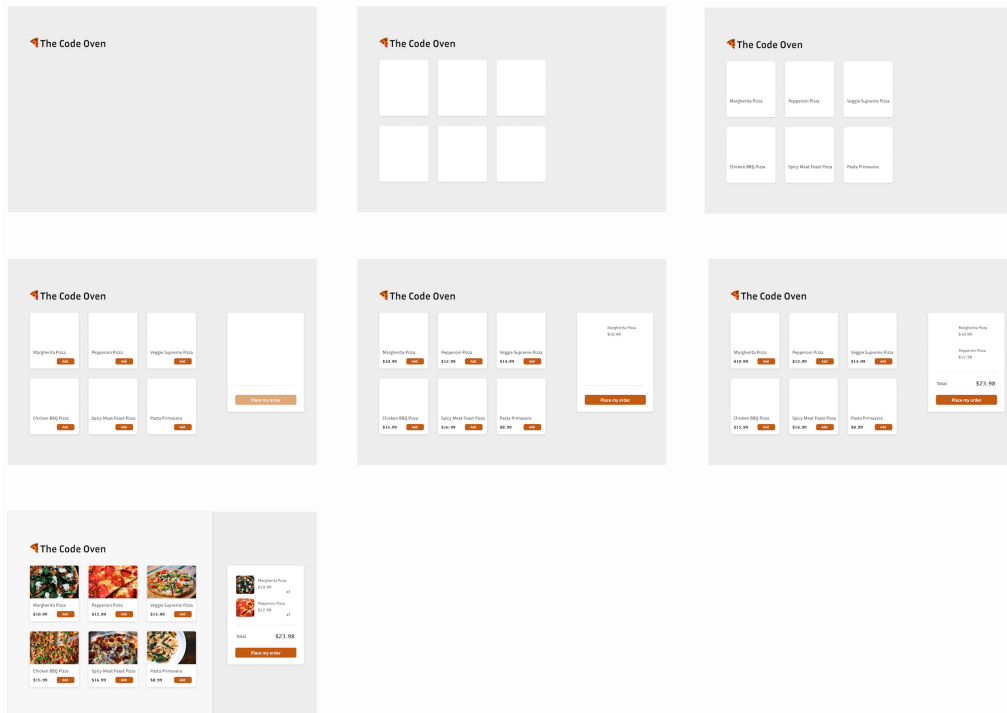


Figure 7.5: The top-down style

(The textual detail in the previous figure is minimized and is not directly relevant to your understanding of it. If you would like to see the textual detail, please refer to the free downloadable eBook.)

Both styles have contributed to the rich variety of methodologies in modern software development, and neither is definitively “right” or “wrong.” Instead, they offer different perspectives and tools that developers can choose from based on their particular needs and preferences.

In the following parts of this chapter, we are going to use the top-down style as it forces us to think from the user’s perspective. We’ll explore the bottom-up approach in more detail in the following chapters when we introduce other design patterns.

Implementing the application headline

Let’s start with the implementation of the pizza store application. If you have cloned the repository mentioned in the *Technical requirements* section, simply go to the `react-anti-patterns-code/src/ch7` folder.

As we’re applying TDD, the first thing to do here is to write a test that fails. In the previous section, we mentioned what we want to test: implement the page title.

So, let’s create a file called `App.test.tsx` with the following code:

```
import React from 'react';
import {render, screen} from '@testing-library/react';

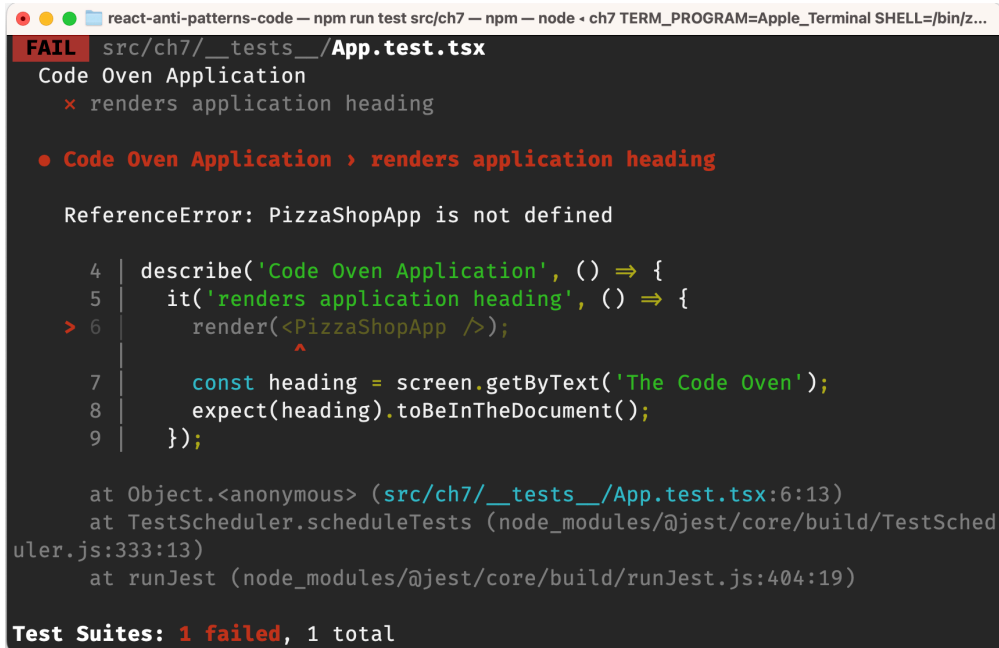
describe('Code Oven Application', () => {
  it('renders application heading', () => {
    render(<PizzaShopApp />);
    const heading = screen.getByText('The Code Oven');
    expect(heading).toBeInTheDocument();
  });
});
```

We’re writing a test for the yet-to-be-created `PizzaShopApp` React component. Using the React Testing Library, we will render this component and verify whether it includes a heading labeled **The Code Oven**. The `expect(heading).toBeInTheDocument();` assertion confirms that the heading is successfully rendered.

Now, let’s run the test with the following command in your terminal window:

```
npm run test src/ch7
```

An error will appear in the terminal saying `ReferenceError: PizzaShopApp is not defined`, like so:



```

react-anti-patterns-code — npm run test src/ch7 — npm — node — ch7 TERM_PROGRAM=Apple_Terminal SHELL=/bin/z...
FAIL src/ch7/__tests__/App.test.tsx
Code Oven Application
  ✕ renders application heading

● Code Oven Application › renders application heading

ReferenceError: PizzaShopApp is not defined

   4 | describe('Code Oven Application', () => {
   5 |   it('renders application heading', () => {
>  6 |     render(<PizzaShopApp />);
      |           ^
   7 |     const heading = screen.getByText('The Code Oven');
   8 |     expect(heading).toBeInTheDocument();
   9 |   });

at Object.<anonymous> (src/ch7/__tests__/App.test.tsx:6:13)
at TestScheduler.scheduleTests (node_modules/@jest/core/build/TestScheduler.js:333:13)
at runJest (node_modules/@jest/core/build/runJest.js:404:19)

Test Suites: 1 failed, 1 total

```

Figure 7.6: The failed test

We're in the red stage of the Red-Green-Refactor loop now, so we need to make the code pass using the simplest code possible. A static component that returns **The Code Oven** would be simple enough to make the test pass. This means we can simply define a function component, `PizzaShopApp`, in the test file that returns just the string:

```

import React from 'react';
import {render, screen} from '@testing-library/react';

function PizzaShopApp() {
  return <>The Code Oven</>;
}

describe("Code Oven Application", () => {
  it("renders application heading", () => {
    render(<PizzaShopApp />);
    const heading = screen.getByText("The Code Oven");
    expect(heading).toBeInTheDocument();
  });
});

```

When we re-run the test, it passes since `PizzaShopApp` does exactly the thing the test expects – it shows **The Code Oven**. Now, we're at the green stage of the Red-Green-Refactor loop. Next, we can look into opportunities for improvement.

We don't want to write all our code inside the test file – instead, we can use the **Move Function** refactoring method we learned about in the previous chapter and move `PizzaShopApp` into a separate file called `App.tsx`. Now, the implementation lives in its own file, allowing us to change the test and the component separately:

```
import React from "react";

export function PizzaShopApp() {
  return <>The Code Oven</>;
}
```

Awesome! With that, we have done a complete Red-Green-Refactor loop. We can now remove that task from our task list and move on to the next one.

Note

The code doesn't have to be perfect at the beginning as we know TDD is an iterative process; we always have a chance to make the code better as we have good tests protecting us.

Implementing the menu list

Even a basic menu list featuring just the names of pizzas can be valuable for customers who are looking to browse and decide what to eat. While The Code Oven may not be set up for online ordering yet, it serves as a useful starting point.

Looking at the second task on our list, we can write our second test like so:

```
it("renders menu list", () => {
  render(<PizzaShopApp />);
  const menuList = screen.getByRole('list');
  const menuItems = within(menuList).getAllByRole('listitem');

  expect(menuItems.length).toEqual(8);
});
```

This test starts by rendering the component. Then, it identifies the HTML element tagged with the `list` role from the rendered component. Using the `within` function, it narrows down the search to only that list and locates all items within it tagged with `listitem`. Finally, it asserts that the number of such items should be equal to 8 (the number of items the pizza shop has to offer). Essentially, we want eight list items to show up on the page.

Now, the test has failed. To make the test pass easily, we can hard-code eight empty list items on the page:

```
import React from "react";

export function PizzaShopApp() {
  return <>
    <h1>The Code Oven</h1>
    <ol>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
    </ol>
  </>;
}
```

It doesn't look too good, but it makes the test pass all the same. This is important to remember – during TDD, we always want to make the test pass first and then look for opportunities to improve afterward. The advantage of this mindset forces us to think about delivery and production readiness; we should be able to stop coding at any point and release the app to production – even if the code isn't perfect yet.

Now, re-run the test to see if it passes; if so, we can start refactoring it. To reduce the long hard-coded `<li>` (the list item tag in HTML), we can use an array with eight elements and use `map` to generate the `<li>` dynamically inside an ordered list (`<ol>`):

```
import React from "react";

export function PizzaShopApp() {
  return <>
    <h1>The Code Oven</h1>
    <ol>
      {new Array(8).fill(0).map(x => <li></li>)}
    </ol>
  </>;
}
```

Inside this list, an array with eight elements (all initialized to 0) is mapped over, generating eight empty list items (`<li>`). This matches the test criteria of having a menu list with eight items and the tests pass with the new structure.

With that, we've done another Red-Green-Refactor loop. Now, we can verify that the pizza names are displayed correctly. Let's add a few more lines to the second test case:

```
it("renders menu list", () => {
  render(<PizzaShopApp />);
  const menuList = screen.getByRole('list');
  const menuItems = within(menuList).getAllByRole('listitem');

  expect(menuItems.length).toEqual(8);

  expect(within(menuItems[0]).getByText('Margherita Pizza')).
    toBeInTheDocument();
  expect(within(menuItems[1]).getByText('Pepperoni Pizza')).
    toBeInTheDocument();
  expect(within(menuItems[2]).getByText('Veggie Supreme Pizza')).
    toBeInTheDocument();
  //...
});
```

To make all the newly added lines pass, we'll need to define a list of pizza names in `PizzaShopApp`, and then use the `map` function in the `pizzas` array to map through these names into list items:

```
const pizzas = [
  "Margherita Pizza",
  "Pepperoni Pizza",
  "Veggie Supreme Pizza",
  "Chicken BBQ Pizza",
  "Spicy Meat Feast Pizza",
  "Pasta Primavera",
  "Caesar Salad",
  "Chocolate Lava Cake"
];

export function PizzaShopApp() {
  return <>
    <h1>The Code Oven</h1>
    <ol>
      {pizzas.map((x) => <li>{x}</li>)}
    </ol>
  </>;
}
```

The tests are now successfully passing. While the code may be overly simplified, the passing tests give us the confidence to make further changes without having to worry about any accidental feature breaks.

It's great to have a menu list, but the purpose of The Code Oven is to help users order online. So, let's look into how we can create a shopping cart.

Creating the shopping cart

To develop a `ShoppingCart` component, we will start with a simple test that expects an empty container to show on the page. Inside the container, there should be a button for the user to place their order.

To do this, we will start with a test that simply checks that the container and button are present:

```
it('renders a shopping cart', () => {
  render(<PizzaShopApp />);

  const shoppingCartContainer = screen.getByTestId('shopping-cart');
  const placeOrderButton = within(shoppingCartContainer).
    getByRole('button');

  expect(placeOrderButton).toBeInTheDocument();
})
```

The Jest test renders the `PizzaShopApp` component and then locates a shopping cart container by its `data-testid`. Within this container, it looks for a button element by its role. The test concludes by using the `toBeInTheDocument()` matcher to verify that this button is present in the rendered output.

To make this test pass, we can add an empty `div` as a container with `data-testid`, and put an empty button inside it:

```
export function PizzaShopApp() {
  return <>
    <h1>The Code Oven</h1>
    <ol>
      {pizzas.map((x) => <li>{x}</li>)}
    </ol>

    <div data-testid="shopping-cart">
      <button></button>
    </div>
  </>;
}
```

As the tests pass, we can add more details to the test, checking if the button is disabled by default.

Note

Observe how we oscillate between the test code and the actual implementation, especially in the beginning. As you become more accustomed to the Red-Green-Refactor cycle, you'll be able to write increasingly complex tests and adjust your code to pass them. The key objective initially is to establish this rapid feedback loop.

We should now add a few more details to the test, to check the button text and the disabled status by default – we want to make sure users cannot interact with the button:

```
it('renders a shopping cart', () => {
  render(<PizzaShopApp />);

  const shoppingCartContainer = screen.getByTestId('shopping-cart');
  const placeOrderButton = within(shoppingCartContainer).
    getByRole('button');

  expect(placeOrderButton).toBeInTheDocument();
  expect(placeOrderButton).toHaveTextContent('Place My Order');
  expect(placeOrderButton).toBeDisabled();
})
```

With the new assertions added, the test failed again, waiting for us to add more details to the implementation. It's straightforward to make the test pass by adding the text and disabled status:

```
export function PizzaShopApp() {
  return <>
    <h1>The Code Oven</h1>
    <ol>
      {pizzas.map((x) => <li>{x}</li>)}
    </ol>

    <div data-testid="shopping-cart">
      <button disabled>Place My Order</button>
    </div>
  </>;
}
```

The tests are all passing again, so that's another task to tick off (note how maintaining a task list can help us focus and gradually shape our application code).

Next, we will look at the next task – adding items from the menu to the shopping cart.

Adding items to the shopping cart

Once we have the basic structure of the `ShoppingCart` component, we need to add a few more assertions to verify it works. We will start by adding one item to the cart, which can be done with the following code:

```
it('adds menu item to shopping cart', () => {
  render(<PizzaShopApp />);

  const menuList = screen.getByRole('list');
  const menuItems = within(menuList).getAllByRole('listitem');

  const addButton = within(menuItems[0]).getByRole('button');
  userEvent.click(addButton);

  const shoppingCartContainer = screen.getByTestId('shopping-cart');
  const placeOrderButton = within(shoppingCartContainer).
    getByRole('button');

  expect(within(shoppingCartContainer).getByText('Margherita Pizza')).
    toBeInTheDocument();
  expect(placeOrderButton).toBeEnabled();
})
```

This test renders the `PizzaShopApp` component, fetches the menu list, and grabs all the list items within it. Then, it simulates a user clicking the **Add** button of the first menu item. Next, it locates the shopping cart container and checks two things:

- The added item, **Margherita Pizza**, appears in the cart
- The **Place My Order** button is enabled

Let's add the button to a menu item first, and then add a state to manage the user selection and enable the **Place My Order** button according to the selection:

```
export function PizzaShopApp() {
  const [cartItems, setCartItems] = useState<string[]>([]);

  const addItem = (item: string) => {
    setCartItems([...cartItems, item]);
  }

  return <>
    <h1>The Code Oven</h1>
    <ol>
```

```

      {pizzas.map((x) => <li>
        {x}
        <button onClick={() => addItem(x)}>Add</button>
      </li>)}
    </ol>

    <div data-testid="shopping-cart">
      <ol>
        {cartItems.map(x => <li>{x}</li>)}
      </ol>
      <button disabled=>Place My Order</button>
    </div>
  </>;
}

```

The `PizzaShopApp` function component uses React's `useState` Hook to manage an array of `cartItems`. It defines a function, `addItem`, to add items to this cart. The component renders a list of pizzas, each with an **Add** button. Clicking the **Add** button invokes the `addItem` function, adding the corresponding pizza to the `cartItems` array.

The shopping cart displays the items in `cartItems` in a list. The **Place My Order** button's disabled state is controlled by the length of the `cartItems` array. Specifically, the button is disabled when the cart is empty – (`cartItems.length === 0`).

The implementation looks great, but if we run the tests, something weird happens. The test failed in the terminal with the following error message: `TestingLibraryElementError: Found multiple elements with the role "list".` This is because we have two lists (one in the menu and the other in the shopping cart) on the screen now, and the React Testing Library is confused about which one should look for the **Add** button, and which one should look for the added items. We can fix that by adding a more specific `data-testid` to the menu list and modifying the test accordingly.

Firstly, let's change our `PizzaShopApp` component and move the first `<ol>` (the ordered list tag) into a `div` element attributed with `data-testid="menu-list"`:

```

<div data-testid="menu-list">
  <ol>
    {pizzas.map((x) => <li>
      {x}
      <button onClick={() => addItem(x)}>Add</button>
    </li>)}
  </ol>
</div>

```

Then, we must modify the test so that it looks as follows (note that we explicitly ask the React Testing Library to search inside `menu-list` for all list items):

```
it('adds menu item to shopping cart', () => {
  render(<PizzaShopApp />);

  const menuItems = within(screen.getByTestId('menu-list')).
    getAllByRole('listitem');

  const addButton = within(menuItems[0]).getByRole('button');
  userEvent.click(addButton);

  const shoppingCartContainer = screen.getByTestId('shopping-cart');
  const placeOrderButton = within(shoppingCartContainer).
    getByRole('button');

  expect(within(shoppingCartContainer).getByText('Margherita Pizza')).
    toBeInTheDocument();
  expect(placeOrderButton).toBeEnabled();
})
```

When we re-run the test, it fails again with this message: `TestingLibraryElementError: Unable to find an element with the text: Margherita Pizza. This could be because the text is broken up by multiple elements. In this case, you can provide a function for your text matcher to make your matcher more flexible.`

The expected **Margherita Pizza** doesn't show up in the shopping cart. This is because we use an event to trigger React to re-render by modifying the `cartItems` state. But when React detects the state change and re-renders, the test isn't waiting for that to happen. In other words, it's too early for the test to see the updated `cartItems`. We need to give React a bit of time to digest the change and re-render. We can mark the test case as `async` and wait for `userEvent.click` to make the state change:

```
it('adds menu item to shopping cart', async () => {
  render(<PizzaShopApp />);

  const menuItems = within(screen.getByTestId('menu-list')).
    getAllByRole('listitem');

  const addButton = within(menuItems[0]).getByRole('button');
  await userEvent.click(addButton);

  const shoppingCartContainer = screen.getByTestId('shopping-cart');
  const placeOrderButton = within(shoppingCartContainer)
    .getByRole('button');

  expect(within(shoppingCartContainer).getByText('Margherita Pizza')).
```

```
    toBeInTheDocument() ;  
    expect(placeOrderButton).toBeEnabled();  
  })
```

In this code snippet, the use of `async` and `await` ensures that the asynchronous operations are completed before they move on to the next step in the test. The test function itself is marked as `async`, making it return a `Promise`, which Jest will wait for before considering the test complete.

The `await userEvent.click(addButton);` line is particularly important here. `userEvent.click` simulates a real user clicking a button and might trigger state updates or effects in your React component. Using `await` ensures that all associated updates and effects are fully completed before moving on to the subsequent lines of the test.

By ensuring that `userEvent.click` has been fully processed, the test will then safely proceed to query and assert against the updated DOM or state. This is crucial for preventing false negatives, where the test might fail not because the code is incorrect, but because the test is checking the DOM before all updates have occurred.

Since all the tests are passing again, it's time to look into other opportunities for improvement.

Refactoring the code

The code we have now isn't too hard to understand, but it has some room for improvement. Let's quickly look at what we have got so far:

```
export function PizzaShopApp() {  
  const [cartItems, setCartItems] = useState<string[]>([]);  
  
  const addItem = (item: string) => {  
    setCartItems([...cartItems, item]);  
  }  
  
  return <>  
    <h1>The Code Oven</h1>  
    <div data-testid="menu-list">  
      <ol>  
        {pizzas.map((x) => <li>  
          {x}  
          <button onClick={() => addItem(x)}>Add</button>  
        </li>)}  
      </ol>  
    </div>  
  
    <div data-testid="shopping-cart">
```

```

    <ol>
      {cartItems.map(x => <li>{x}</li>)}
    </ol>
    <button disabled={cartItems.length === 0}>Place My Order
    </button>
  </div>
</>;
}

```

Now, let's make some changes. Firstly, we could use the **Rename Variable** refactoring to change `x` to `item`, to make it a bit more meaningful. Also, there is a warning in the terminal now, saying **Warning: Each child in a list should have a unique "key" prop** – as React expects a unique key for each item it renders, we need to have a key for each `<li>` element. For now, we can use the `item` (the pizza name) as the key to fix this issue:

```

export function PizzaShopApp() {
  const [cartItems, setCartItems] = useState<string[]>([]);

  const addItem = (item: string) => {
    setCartItems([...cartItems, item]);
  }

  return <>
    <h1>The Code Oven</h1>
    <div data-testid="menu-list">
      <ol>
        {pizzas.map((item) => <li key={item}>
          {item}
          <button onClick={() => addItem(item)}>Add</button>
        </li>)}
      </ol>
    </div>

    <div data-testid="shopping-cart">
      <ol>
        {cartItems.map(item => <li key={item}>{item}</li>)}
      </ol>
      <button disabled={cartItems.length === 0}>Place My Order
      </button>
    </div>
  </>;
}

```


As another change, the menu item list looks pretty self-contained and doesn't depend on anything out of the context it lives in, so we can extract a new component here to encapsulate that logic:

```
const MenuItemList = ({
  onAddMenuItem,
}: {
  onAddMenuItem: (item: string) => void;
}) => {
  return (
    <div data-testid="menu-list">
      <ol>
        {pizzas.map((item) => (
          <li key={item}>
            {item}
            <button onClick={() => onAddMenuItem(item)}>Add</button>
          </li>
        ))}
      </ol>
    </div>
  );
};
```

The `MenuItemList` component takes a single prop, `onAddMenuItem`, which is a function that accepts a string argument representing a menu item. The component renders a list of pizzas, which presumably is an array of strings. For each pizza, it creates a list item and an **Add** button. When this button is clicked, the `onAddMenuItem` function is called with the corresponding pizza name as an argument. The component uses the `data-testid="menu-list"` attribute to make it easier to query this section during testing. Overall, this is a presentational component that's designed to display a list of pizzas and handle the addition of menu items through the provided callback.

Similarly, we can extract a new component for the shopping cart, like so:

```
const ShoppingCart = ({ cartItems }: { cartItems: string[] }) => {
  return (
    <div data-testid="shopping-cart">
      <ol>
        {cartItems.map((item) => (
          <li key={item}>{item}</li>
        ))}
      </ol>
      <button disabled={cartItems.length === 0}>Place My Order
    </button>
  );
};
```

```
    </div>
  );
};
```

The `ShoppingCart` component takes an array of `cartItems` as its prop. This array holds the names of the items that have been added to the shopping cart. The component renders an ordered list (`<ol>`) where each list item (`<li>`) corresponds to an item in the cart. It uses the `data-testid="shopping-cart"` attribute to make it easier to identify this component in tests. Additionally, a **Place My Order** button is rendered at the bottom of the list. The button is disabled if the `cartItems` array is empty, meaning there are no items in the cart. Overall, this component is designed to display the items in the shopping cart and provide an option to place an order.

After these extractions, we can use these components in the main component, `PizzaShopApp`:

```
export function PizzaShopApp() {
  const [cartItems, setCartItems] = useState<string[]>([]);

  const addItem = (item: string) => {
    setCartItems([...cartItems, item]);
  };

  return (
    <>
      <h1>The Code Oven</h1>
      <MenuList onAddMenuItem={addItem} />
      <ShoppingCart cartItems={cartItems} />
    </>
  );
}
```

During these relatively big changes, our tests always stay in the green state – meaning no functions are broken. We haven't implemented all the tasks we have broken down, but I believe you have got the idea of the Red-Green-Refactor loop. You can use the remaining tasks as exercises, making sure you always write the test first, with only the minimal code, and looking for improvements after the test passes.

You may initially find this coding approach challenging as it requires you to resist the temptation to immediately delve into implementation. Instead, take incremental steps and savor the journey. You'll soon discover the strength that lies in taking small steps, as well as how maintaining a steady rhythm can enhance your focus and productivity.

Summary

In this chapter on TDD, we've explored various forms of TDD, emphasizing the importance of breaking down complex problems into manageable tasks through tasking. We delved into two key approaches – top-down and bottom-up, each with its distinct merits and use cases. To illustrate these concepts, we used a practical example of building a pizza shop application.

This hands-on example helped solidify the theories and methodologies we discussed, providing a comprehensive understanding of how TDD can be employed effectively in different scenarios.

In the upcoming chapter, we'll delve deeper into the intricate world of data management in React applications. Specifically, we'll explore various design patterns that are commonly adopted for efficient data access and manipulation.

Part 3:

Unveiling Business Logic and Design Patterns

This part looks at business logic and design patterns, which are essential in tackling common challenges in state management and adhering to principles, such as the single responsibility principle, in order to maintain a clean and efficient code base.

This part contains the following chapters:

- *Chapter 8, Exploring Data Management in React*
- *Chapter 9, Applying Design Principles in React*
- *Chapter 10, Diving Deep into Composition Patterns*

8

Exploring Data Management in React

In modern frontend development, the way we handle state and data access can make or break an application. Whether you're a solo developer or part of a large team, understanding best practices and common pitfalls is critical. This chapter aims to elevate your proficiency in state management and data handling within React applications, focusing on scalable and maintainable approaches.

State management is challenging, particularly in React, and there are many issues that developers tend to face in their day-to-day work. One of the challenges is where you can locate the business logic in your code base. When business logic infiltrates UI components, it compromises their reusability. Many domain objects, along with computational logic, are scattered in UI components either deliberately or unintentionally, which can lead to tangled logic that's hard to follow, debug, and test. It can also lead to performance issues, which negatively affect the user experience.

Another issue is prop drilling, where passing props from a parent component to deeply nested children becomes both cumbersome and susceptible to errors. This often leads to code duplication as the same snippets find their way into multiple files, making any future updates a complicated task.

Lastly, sharing state in a React application presents its own set of challenges. Various mechanisms exist for this, but choosing the most efficient way to share stateful logic across components can be quite perplexing. We'll dive deep to explore the issues of sharing state and see how React's Context API can help.

In this chapter, we will cover the following topics:

- Understanding business logic leaks
- Introducing the anti-corruption layer
- Exploring the prop drilling issue
- Using the Context API to resolve prop drilling

Technical requirements

A GitHub repository has been created to host all the code we'll discuss in this book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch8>.

Understanding business logic leaks

Business logic refers to the rules, calculations, and processes that are essential to the operation of a business application. When this business logic “leaks” into components or areas of the application where it doesn't belong, this is known as a **business logic leak**.

This issue frequently crops up in various projects, partly because there's no widely agreed-upon approach for handling business logic in React. The framework's flexibility allows you to implement this logic directly in components, Hooks, or helper functions; as a result, developers often end up embedding the logic directly into components, where it's immediately needed – hence the leakage.

This leakage can cause many problems. Business logic leaks can result in tightly coupled components that become difficult to test, maintain, or reuse. When business logic is scattered across different parts of the application, it leads to code duplication and inconsistency, making the application more prone to errors and harder to debug. Additionally, this scattering complicates any future modifications to the business rules as changes will likely have to be made in multiple places, increasing the risk of introducing new issues.

There are various indicators that business logic is leaking into your code, but the most prevalent sign is the incorporation of data transformation directly within views or UI components. In this section, we'll delve into this issue in depth; in the following section, we'll explore solutions to mitigate this problem.

Data transformation is a function that takes one shape (or format) of data (usually from another module or a remote service) and maps the data to the shape that fits another usage. The following is a typical example you would see in many React applications, showing a `UserProfile` function:

```
function UserProfile({ id }: { id: string }) {  
  const [user, setUser] = useState<User | null>(null);  
  
  useEffect(() => {
```

```
async function fetchUser() {
  const response = await fetch(`/api/users/${id}`);
  const data = await response.json();

  setUser({
    id: data.user_identification,
    name: data.user_full_name,
    isPremium: data.is_premium_user,
    subscription: data.subscription_details.level,
    expire: data.subscription_details.expiry,
  });
}

fetchUser();
}, [id]);

if (!user) {
  return <div>Loading...</div>;
}

return (
  <div data-testid="user-profile">
    <h1>{user.name}</h1>
  </div>
);
}
```

The `UserProfile` function component fetches user data from a backend API. The `useEffect` Hook ensures that data fetching happens whenever the ID changes. The fetched data is then stored in a local state variable, `user`, and the component displays the user's name once the data is available. If the data is still loading, a **Loading...** message is displayed.

In the `useEffect` block, the fetched data transforms being stored in the `user` state variable. Specifically, the keys from the fetched JSON response are mapped to new names that are more suitable for the frontend application – for example, `data.user_identification` becomes `id` and `data.user_full_name` becomes `name`. This transformation allows for easier handling and readability within the React component.

As you might imagine, this type of transformation can happen in many places in an application, and it isn't limited to the inside of a React component; sometimes, this transformation can be present in Hooks and other places, as demonstrated in the following figure:

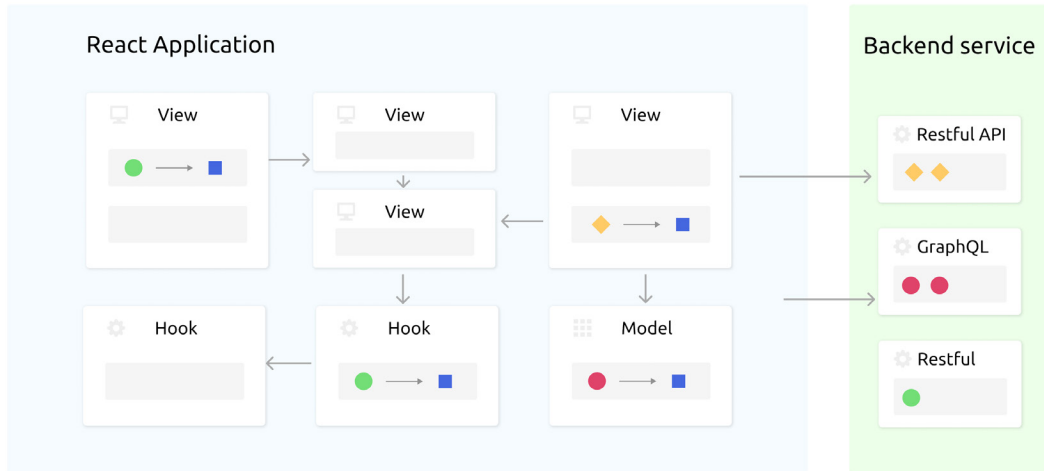


Figure 8.1: Using data transformation in views

The backend service can serve data in various formats that are supported by different technologies – some utilize RESTful APIs, while others employ GraphQL. From the perspective of the frontend code, the specifics of these formats are abstracted away. For instance, one component may be responsible for transforming XML data (indicated by an orange diamond shape) into an internal type, *X* (depicted as a blue square). Meanwhile, another component may interact with a GraphQL endpoint, processing the received data (represented by a red circle) into the same internal type, *X*.

This discrepancy necessitates data transformations across multiple areas in the code. When such transformations are duplicated, there's a higher risk of overlooking changes, especially if the backend alters the structure of their data.

All these varied transformations can be centralized into a single location where data reshaping occurs, along with empty field checks, field renaming, and the elimination of unnecessary fields. This segues nicely into our next topic: the **anti-corruption layer (ACL)**.

Introducing the ACL

In software development, an ACL acts like a translator or a mediator between different subsystems that may not speak the same “language,” so to speak. Imagine that you have two systems, each with its own set of rules, structures, and complexities. If these systems interact directly, there's a risk that they could influence each other in unintended ways, leading to what is called *corruption* in the domain logic.

In the context of frontend development, especially in complex applications, an ACL becomes crucial for managing the interactions between the frontend and various backends or APIs. Frontend developers often have to deal with multiple services that may have inconsistent or convoluted data formats. Implementing an ACL in the frontend allows you to create a unified interface to interact with these services.

For example, if your frontend application has to communicate with multiple RESTful APIs, GraphQL services, and even WebSocket servers, each may have its own set of rules, data structures, and complexities. A frontend ACL would take on the role of translating these disparate forms of data into a format that your frontend application understands. This means that your UI components don't have to worry about the intricate details of each service's data format, making the components easier to develop, test, and maintain.

The ACL can also be a strategic place to handle caching, error transformations, and other cross-cutting concerns. By centralizing these functionalities, you avoid scattering similar logic all over your frontend code base, thus adhering to the **Don't Repeat Yourself (DRY)** principle:

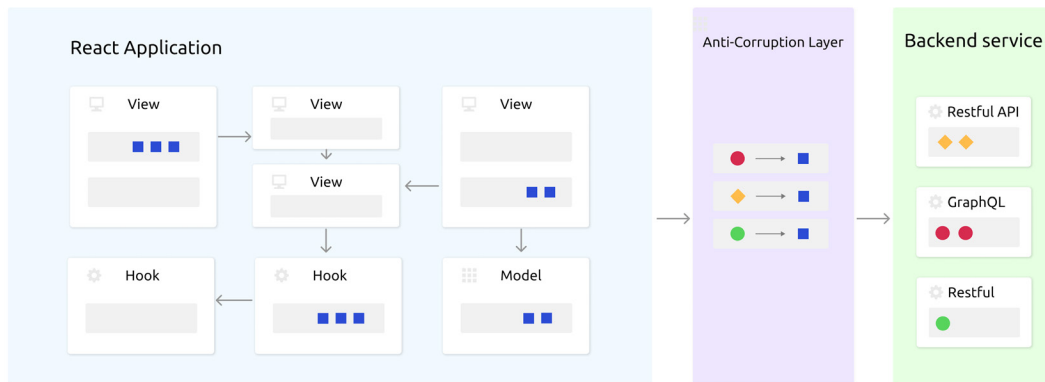


Figure 8.2: Introduced ACL for data transforming

As illustrated in *Figure 8.2*, all data transformations are now centralized, eliminating the need for such operations in the views. You might be curious about how to put this into practice in your code.

Introducing a typical usage

To get started, let's build on the example from the previous section and introduce a basic function as the starting point for our ACL. We should identify the external data format and what we consume, then define functions as the transformers before putting them in a common place.

First, let's define a type for the user data format we receive from the remote server. By using TypeScript, we gain the advantage of compile-time checks, thereby ensuring that any inconsistencies in the data format are flagged before the application runs:

```
type RemoteUser = {
  user_identification: string;
  user_full_name: string;
  is_premium_user: boolean;
  subscription_details: {
    level: string;
```

```
    expiry: string;
  }
}
```

We will also define the local `User` type – it includes all fields (`id`, `name`, and so on) that we use in the `UserProfile` component:

```
type UserSubscription = "Basic" | "Standard" | "Premium" |
  "Enterprise";

type User = {
  id: string;
  name: string;
  isPremium: boolean;
  subscription: UserSubscription;
  expire: string;
};
```

The type definition specifies the structure of a user object that will be used in React components. Note that the `subscription` property uses a custom type, `UserSubscription`, which can take one of four string values: **Basic**, **Standard**, **Premium**, or **Enterprise**. Also, `expire` is a string that indicates when the user's subscription will expire.

With this setup, we can define a new function called `transformUser` in a file called `transformer.ts`. The file simply returns the mapped object with the `User` type we just defined:

```
import {RemoteUser, User, UserSubscription} from "./types";

export const transformUser = (remoteUser: RemoteUser): User => {
  return {
    id: remoteUser.user_identification,
    name: remoteUser.user_full_name,
    isPremium: remoteUser.is_premium_user,
    subscription: remoteUser.subscription_details.level as
      UserSubscription,
    expire: remoteUser.subscription_details.expiry,
  };
};
```

With this new extracted function in `transformer.ts`, our component can be simplified like so:

```
async function fetchUserData<T>(id: string) {
  const response = await fetch(`/api/users/${id}`);
  const rawData = await response.json();

  return transformUser(rawData) as T;
}
```

```
}

function UserProfile({ id }: { id: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    async function fetchUser() {
      const response = await fetchUserData<User>(id);
      setUser(response);
    }

    fetchUser();
  }, [id]);

  if (!user) {
    return <div>Loading...</div>;
  }

  return (
    <div data-tested="user-profile">
      <h1>{user.name}</h1>
    </div>
  );
}
```

The `UserProfile` component relies on the `fetchUserData` helper function to retrieve and process user data from an API. This design insulates `UserProfile` from any knowledge of the remote data structure. If there are future changes to the `RemoteUser` type, `UserProfile` remains unaffected as all the adjustments will be confined to `transformer.ts`.

It's advantageous to have a distinct function dedicated to managing remote data and molding it to suit the requirements of higher-level views. However, complications arise when the backend fails to deliver the necessary data. In such scenarios, additional logic is necessitated in this layer to establish fallback or default values.

Using the fallback or default value

Another frequently observed issue related to data transformation is the excessive use of defensive programming in React views. While defensive programming is generally good practice and useful in various contexts, overloading React components with too many null checks and fallbacks can clutter the code and make it difficult to comprehend.

Note

Defensive programming is a practice that involves writing code in a way that anticipates possible errors, failures, or exceptions, and handles them gracefully. The goal is to make your application more resilient and maintainable by minimizing the impact of unexpected scenarios.

For instance, in the `UserProfile` example, you might encounter situations where certain values from the remote service are empty. Instead of displaying `null` or `undefined` to end users, you would need to implement fallback values.

Let's review the `transformUser` function we extracted previously:

```
export const transformUser = (remoteUser: RemoteUser): User => {
  return {
    id: remoteUser.user_identification,
    name: remoteUser.user_full_name,
    isPremium: remoteUser.is_premium_user,
    subscription: remoteUser.subscription_details.level as
      UserSubscription,
    expire: remoteUser.subscription_details.expiry,
  };
};
```

What if `subscription_details` doesn't exist, or what happens when `expiry` isn't a valid date format from the backend? These mismatches can cause runtime exceptions, so we should fall back to some default value when the remote data isn't in the right format.

We could put the fallback logic into components, right before we render them. Without the ACL, we might end up with some logic in `UserProfile`, like this:

```
function UserProfile({ user }: { user: User }) {
  const fullName = user && user.name ? user.name : "Loading...";
  const subscriptionLevel =
    user && user.subscription ? user.subscription : "Basic";
  const subscriptionExpiry = user && user.expiry ? user.expiry :
    "Never";

  return (
    <div>
      <h1>{fullName}</h1>
      <p>Subscription Level: {subscriptionLevel}</p>
      <p>Subscription Expiry: {subscriptionExpiry}</p>
    </div>
  );
}
```

The `UserProfile` function component takes a user object as a prop. It displays the user's full name, subscription level, and subscription expiry date. If any of these values are missing or falsy, it provides default fallback text such as **Loading...**, **Basic**, or **Never**.

As such, logic begins to infiltrate the `UserProfile` component, increasing the component's length and therefore complexity.

This kind of logic can be better managed by moving it into a function such as `transformUser`, where it can be thoroughly tested:

```
export const transformUser = (remoteUser: RemoteUser): User => {
  return {
    id: remoteUser.user_identification ?? 'N/A',
    name: remoteUser.user_full_name ?? 'Unknown User',
    isPremium: remoteUser.is_premium_user ?? false,
    subscription: (remoteUser.subscription_details?.level ?? 'Basic')
      as UserSubscription,
    expire: remoteUser.subscription_details?.expiry ?? 'Never',
  };
};
```

The `transformUser` function maps fields from the remote user data structure to the application's expected user data structure, providing default values for each field in case they are missing or null. For example, if `remoteUser.user_identification` is null, it will use **N/A** as the default ID.

Note that here, we use **optional chaining** (`?`), which allows you to access deeply nested properties without checking each level of nesting. If `subscription_details` or `level` is null or undefined, the result, `subscription`, will also be undefined and no errors will be thrown. We also used **nullish coalescing** (`??`) for falling back – it takes the value on its left if that value isn't null or undefined; otherwise, it takes the value on its right.

All this transforming and fallback logic is now encapsulated in a common place – the ACL. Any further changes to the remote or local data shape can easily happen in this layer, and we don't have to look up the entire code base for different usages.

Excellent – the ACL pattern effectively isolates business logic from your views. However, there are additional challenges in managing data within a React application, such as sharing data between components and avoiding prop drilling. In the following section, we'll explore how to tackle these issues using the Context API.

Exploring the prop drilling issue

Prop drilling is an issue that arises when you have to pass data through multiple levels of components that don't need the data, just so it can reach a component deeper in the hierarchy that does need it. This often makes the code harder to follow, understand, and maintain.

Consider a standard scenario in React where we have a universal searchable list component. This component takes in a list and displays each item, whether it's a list of books, menus, tickets, or anything else you can think of. In addition to displaying the list, the component also includes a search box, allowing users to easily filter through a lengthy list:

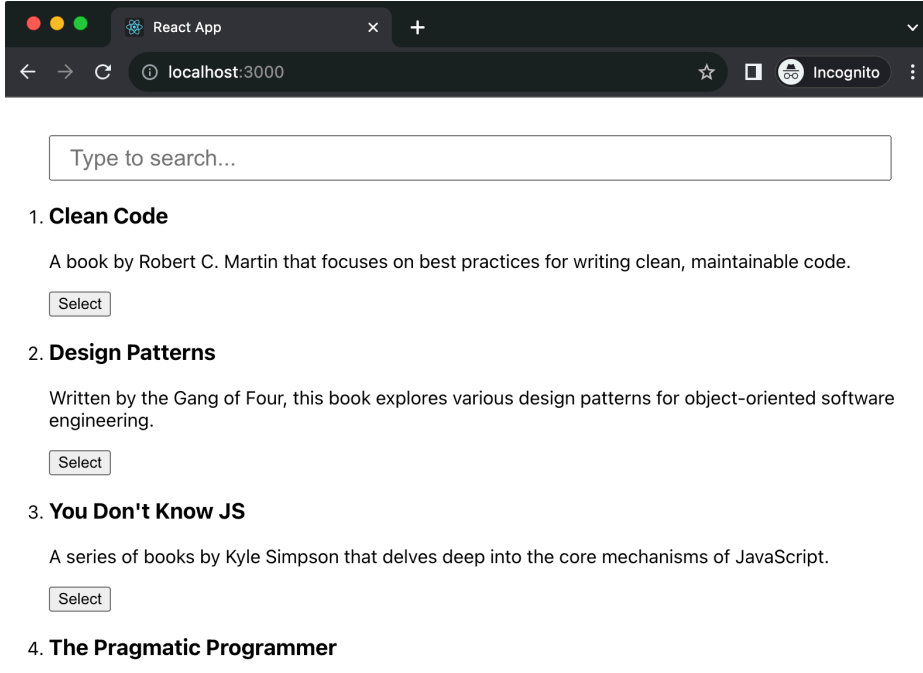


Figure 8.3: The searchable list component

At first glance, the implementation seems straightforward and not overly complicated:

```
import React, { ChangeEvent, useState } from "react";

export type Item = {
  id: string;
  name: string;
  description: string;
};

const SearchableList = ({ items }: { items: Item[] }) => {
  const [filteredItems, setFilteredItems] = useState<Item[]>(items);

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setFilteredItems(
```

```
    items.filter((item) => item.name.includes(e.target.value))
  );
};

return (
  <div>
    <input type="text" onChange={handleChange} />
    <ul>
      {filteredItems.map((item, index) => (
        <li key={index}>{item.name}</li>
      ))}
    </ul>
  </div>
);
};

export default SearchableList;
```

The code defines a `SearchableList` component in React that filters and displays a list of items based on user input. It starts with a full list of items and updates the filtered list whenever the text in the input box changes.

Over time, as the component evolves and is used in more diverse scenarios, the code base becomes increasingly complex, resulting in additional layout changes and more lines of code. As illustrated in *Figure 8.4*, on the right-hand side, we can break the searchable list input into three sub-components called `SearchInput`, `List`, and `ListItem`:

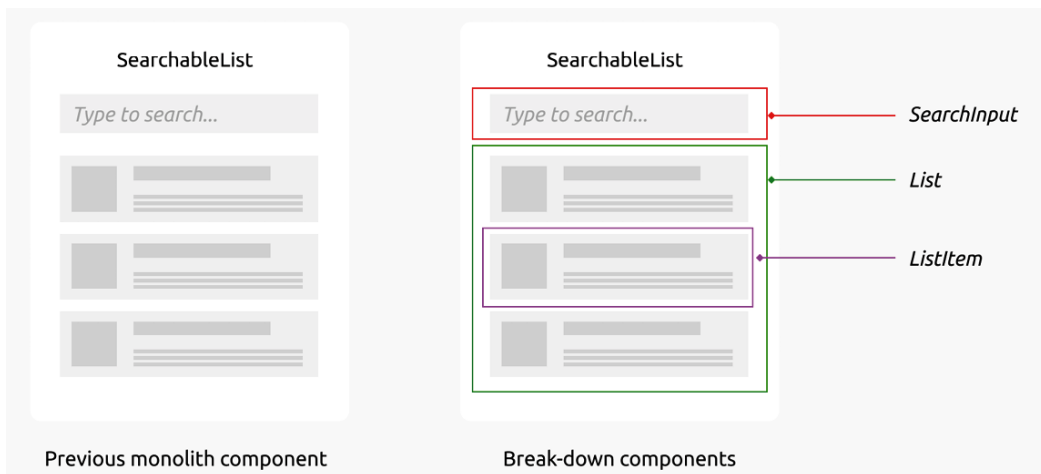


Figure 8.4: Breaking a searchable list into smaller components

On the right-hand side, `SearchInput` accepts the user's input (in the top red rectangle). It is a sibling of the `List` component (inside the green rectangle), which contains multiple `ListItem` (in the purple rectangle) – each `ListItem` represents an item (one might have a title, description, or buttons in it).

Let's look at each of these extracted components in detail:

```
const ListItem = ({ item }: { item: Item }) => {
  return (
    <li>
      <h2>{item.name}</h2>
      <p>{item.description}</p>
    </li>
  );
};
```

`List` displays the name and description of an item. For any further details that are needed, we only need to modify this file directly.

Then, each `List` is wrapped inside a container component, `List`, like so:

```
const List = ({ items }: { items: Item[] }) => {
  return (
    <section data-testid="searchable-list">
      <ul>
        {items.map((item) => (
          <ListItem item={item} />
        ))}
      </ul>
      <footer>Total items: {items.length}</footer>
    </section>
  );
};
```

This `List` acts as the container for all the items. It also includes a footer to show some summary information of the items.

Finally, we have the `SearchInput` component, which is responsible for collecting user input and triggering searches as the user is typing:

```
const SearchInput = ({ onSearch }: { onSearch: (keyword: string) => void }) => {
  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    onSearch(e.target.value);
  };

  return <input type="text" onChange={handleChange} />;
};
```

```
};

const SearchableList = ({ items }: { items: Item[] }) => {
  const [filteredItems, setFilteredItems] = useState<Item[]>(items);

  const onSearch = (keyword: string) => {
    setFilteredItems(items.filter((item) => item.name.
      includes(keyword)));
  };

  return (
    <div>
      <SearchInput onSearch={onSearch} />
      <List items={filteredItems} />
    </div>
  );
};
```

Note that this code breakdown is only an internal structural change – we made the change only to make each part easy to read and understand. People who use `SearchableList` aren't aware of such changes at all – the props of `SearchableList` haven't been changed so far.

However, the consumer has requested a new feature: they wish to track user interactions for analytics, specifically to gauge the popularity of items and the utilization of the search function. To meet this requirement, two new props are being introduced: an `onItemClicked` callback to capture when an item is clicked, and an `onSearch` callback to monitor when a search is performed.

So, we need to modify our code to meet these requirements, starting from the new type used in `SearchableList`:

```
type SearchableListProps = {
  items: Item[];
  onSearch: (keyword: string) => void;
  onItemClicked: (item: Item) => void;
};

const SearchableList = ({
  items,
  onSearch,
  onItemClicked,
}: SearchableListProps) => {
  //...
}
```

To pass `onSearch` and `onItemClicked`, we'll make some changes to `ListItem`. We need to change the props list (adding `onItemClicked`) and then call the function when the list is clicked:

```
const ListItem = ({
  item,
  onItemClicked,
}: {
  item: Item;
  onItemClicked: (item: Item) => void;
}) => {
  return (
    <li onClick={() => onItemClicked(item)}>
      <h2>{item.name}</h2>
      <p>{item.description}</p>
    </li>
  );
};
```

However, as `ListItem` is not exposed directly to the outside world, the props are passed from its parent component, `List`. So, we also need to update the `List` component's props list to add `onItemClicked`:

```
const List = ({
  items,
  onItemClicked,
}: {
  items: Item[];
  onItemClicked: (item: Item) => void;
}) => {
  return (
    <section data-testid="searchable-list">
      <ul>
        {items.map((item) => (
          <ListItem item={item} onItemClicked={onItemClicked} />
        ))}
      </ul>
      <footer>Total items: {items.length}</footer>
    </section>
  );
};
```

The `List` component has to accept `onItemClicked` as a prop and then pass on the `onItemClicked` prop into `ListItem` without touching it. This is a code smell, signaling that the component is dealing with something not directly relevant to its function.

Then, the `onItemClick` prop is passed from the `List` component's parent `SearchableList` component, as shown here:

```
const SearchableList = ({
  items,
  onSearch,
  onItemClick,
}: SearchableListProps) => {
  const [filteredItems, setFilteredItems] = useState<Item[]>(items);

  const handleSearch = (keyword: string) => {
    setFilteredItems(items.filter((item) => item.name.
      includes(keyword)));
  };

  return (
    <div>
      <SearchInput onSearch={handleSearch} />
      <List items={filteredItems} onItemClick={onItemClick} />
    </div>
  );
};
```

Observe how we transfer the `onItemClick` prop to the `List` component. First, it goes through the `List` component, which directly passes it down to `List` without using it for anything. This is a classic example of prop drilling. The same could happen with `SearchInput`. As we continue to add more and more props from the outside and drill them down through the component tree, the entire structure could soon become unmanageable.

Fortunately, the Context API provides an elegant solution to the prop drilling issue, which we will review in the next section (we covered the fundamentals of using the Context API in *Chapter 2*; you can revisit that chapter if you'd like a refresher).

Using the Context API to resolve prop drilling

The concept behind using the Context API to address prop drilling is to create a shared container for all sub-components under a common parent. This eliminates the need for explicitly passing down props from parent to child. Sub-components can directly access the shared context whenever necessary. Another advantage of using the Context API is that it triggers automatic re-rendering of components whenever the data within the context changes.

Returning to our searchable list example, which has already got the *props drilling* issue, `onItemClick` in the `List` component isn't necessary as `List` doesn't use the prop. This scenario involves just one layer and one prop. Now, envision a situation where we need to insert additional elements between